

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



# TÀI LIỆU HỌC TẬP

## SIMPLE OPERATING SYSTEM SIMULATOR

Từ nền tảng lý thuyết đến giải thích source code và vấn đáp

|                  |                                                                                         |
|------------------|-----------------------------------------------------------------------------------------|
| <b>Môn học:</b>  | Hệ điều hành – CO2018                                                                   |
| <b>Lớp:</b>      | L08                                                                                     |
| <b>Phạm vi:</b>  | Scheduler, synchronization, virtual memory, system call, multi-level paging và kiểm thử |
| <b>Mục đích:</b> | Học lại từ đầu, hiểu bài làm và chuẩn bị trình bày                                      |

TP. Hồ Chí Minh, tháng 06/2026

# Cách sử dụng tài liệu

Tài liệu này không thay thế báo cáo nộp bài. Đây là một “bản đồ học tập” được viết riêng để thành viên nhóm có thể bắt đầu từ kiến thức nền tảng, sau đó lần theo đúng luồng thực thi của repository hiện tại. Mỗi chương trả lời bốn câu hỏi:

1. Khái niệm hệ điều hành nào đang được mô phỏng?
2. Khái niệm đó được hiện thực ở file và cấu trúc dữ liệu nào?
3. Làm sao kiểm chứng implementation đang hoạt động đúng?
4. Nếu giảng viên hỏi sâu hơn, giới hạn và trade-off là gì?

**Điểm cần nhớ.** Không nên học thuộc từng dòng code. Hãy học các *bất biến* (invariant), luồng dữ liệu và lý do thiết kế. Khi hiểu ba thứ đó, ta có thể giải thích code ngay cả khi quên tên một hàm cụ thể.

## Lộ trình học đề xuất

| Buổi | Trọng tâm             | Kết quả cần đạt                                                                                   |
|------|-----------------------|---------------------------------------------------------------------------------------------------|
| 1    | Bức tranh tổng thể    | Vẽ được luồng loader – scheduler – CPU – syscall – memory – timer.                                |
| 2    | Process và scheduler  | Giải thích PCB, ready/running, MLQ, priority, weighted slot và quantum.                           |
| 3    | Đồng bộ               | Phân biệt mutex, condition variable và barrier theo time slot; chỉ ra race condition nếu bỏ lock. |
| 4    | Virtual memory        | Tính được địa chỉ vật lý từ page/frame/offset; giải thích VMA, region, free-region và swap.       |
| 5    | System call và bảo vệ | Mô tả đường đi của syscall 17 và lý do kernel phải lookup PCB bằng PID.                           |
| 6    | MM64                  | Tách địa chỉ canonical 57-bit thành năm index và giải thích sparse page-table allocation.         |
| 7    | Kiểm thử và vấn đáp   | Đọc được log, liên hệ log với invariant, tự trả lời bộ câu hỏi cuối tài liệu.                     |

## Contents

|                                                         |          |
|---------------------------------------------------------|----------|
| Cách sử dụng tài liệu                                   | i        |
| 1 <b>Bức tranh tổng thể của hệ điều hành mô phỏng</b>   | <b>1</b> |
| 1.1 Hệ điều hành thực sự làm gì? . . . . .              | 1        |
| 1.2 Kiến trúc end-to-end . . . . .                      | 1        |
| 1.3 Phân biệt simulation và hệ điều hành thật . . . . . | 1        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>2</b> | <b>Process, PCB, loader và CPU</b>                       | <b>2</b>  |
| 2.1      | Process và program khác nhau thế nào? . . . . .          | 2         |
| 2.2      | PCB là trung tâm của trạng thái process . . . . .        | 2         |
| 2.3      | Loader biến file thành PCB . . . . .                     | 3         |
| 2.4      | CPU fetch–execute như thế nào? . . . . .                 | 3         |
| 2.5      | Vòng đời PCB . . . . .                                   | 3         |
| 2.6      | Câu hỏi tự kiểm tra . . . . .                            | 4         |
| <b>3</b> | <b>CPU Scheduling và Multi-Level Queue</b>               | <b>4</b>  |
| 3.1      | Bài toán scheduler . . . . .                             | 4         |
| 3.2      | MLQ trong repository . . . . .                           | 4         |
| 3.3      | Weighted slot . . . . .                                  | 4         |
| 3.4      | Priority và quantum là hai khái niệm khác nhau . . . . . | 5         |
| 3.5      | Running list giải quyết vấn đề gì? . . . . .             | 5         |
| 3.6      | Bất biến scheduler . . . . .                             | 5         |
| 3.7      | Độ phức tạp và trade-off . . . . .                       | 6         |
| 3.8      | Đọc testcase scheduler . . . . .                         | 6         |
| <b>4</b> | <b>Concurrency, timer và synchronization</b>             | <b>6</b>  |
| 4.1      | Tại sao simulator cần synchronization? . . . . .         | 6         |
| 4.2      | Mutex và condition variable . . . . .                    | 7         |
| 4.3      | Timer barrier theo time slot . . . . .                   | 7         |
| 4.4      | Phạm vi lock và deadlock . . . . .                       | 7         |
| <b>5</b> | <b>System call và ranh giới user/kernel</b>              | <b>8</b>  |
| 5.1      | Tại sao cần system call? . . . . .                       | 8         |
| 5.2      | Đường đi của một memory operation . . . . .              | 8         |
| 5.3      | Unified memory syscall . . . . .                         | 8         |
| 5.4      | Kernel lookup PCB bằng PID . . . . .                     | 8         |
| 5.5      | Bảo vệ raw physical access . . . . .                     | 9         |
| 5.6      | Ưu và nhược điểm của unified syscall . . . . .           | 9         |
| <b>6</b> | <b>Nền tảng quản lý bộ nhớ</b>                           | <b>9</b>  |
| 6.1      | Ba loại địa chỉ cần phân biệt . . . . .                  | 9         |
| 6.2      | VMA, region và symbol table . . . . .                    | 10        |
| 6.3      | ALLOC . . . . .                                          | 10        |
| 6.4      | FREE và tái sử dụng . . . . .                            | 10        |
| 6.5      | READ/WRITE và kiểm tra biên . . . . .                    | 10        |
| 6.6      | Physical memory và free-frame list . . . . .             | 11        |
| 6.7      | Paging so với continuous allocation . . . . .            | 11        |
| <b>7</b> | <b>Page table, PTE và swapping</b>                       | <b>11</b> |
| 7.1      | PTE biểu diễn trạng thái gì? . . . . .                   | 11        |
| 7.2      | Khi page không ở RAM . . . . .                           | 12        |
| 7.3      | FIFO replacement . . . . .                               | 12        |
| 7.4      | Bất biến bộ nhớ . . . . .                                | 12        |
| <b>8</b> | <b>Kernel memory, cache pool và copy user/kernel</b>     | <b>12</b> |
| 8.1      | User memory và kernel memory . . . . .                   | 12        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| 8.2       | KMALLOC . . . . .                                                    | 13        |
| 8.3       | Kernel cache pool . . . . .                                          | 13        |
| 8.4       | Copy qua ranh giới protection . . . . .                              | 13        |
| <b>9</b>  | <b>Multi-Level Paging 64-bit</b>                                     | <b>13</b> |
| 9.1       | Vì sao cần nhiều cấp? . . . . .                                      | 13        |
| 9.2       | Phân rã địa chỉ 57-bit . . . . .                                     | 13        |
| 9.3       | Canonical address . . . . .                                          | 14        |
| 9.4       | Page-table walk và sparse allocation . . . . .                       | 14        |
| 9.5       | Thống kê . . . . .                                                   | 14        |
| 9.6       | Lợi ích và chi phí của N-level paging . . . . .                      | 15        |
| <b>10</b> | <b>Đọc source code theo luồng thay vì theo tên file</b>              | <b>15</b> |
| 10.1      | Luồng khởi động . . . . .                                            | 15        |
| 10.2      | Luồng một instruction ALLOC . . . . .                                | 15        |
| 10.3      | Luồng một instruction READ . . . . .                                 | 15        |
| 10.4      | Luồng process hết quantum và kết thúc . . . . .                      | 16        |
| <b>11</b> | <b>Kiểm thử có ý nghĩa</b>                                           | <b>16</b> |
| 11.1      | Ba tầng kiểm thử . . . . .                                           | 16        |
| 11.2      | Traceability từ yêu cầu đến test . . . . .                           | 16        |
| 11.3      | Cách đọc log . . . . .                                               | 16        |
| 11.4      | Lệnh kiểm chứng khi học . . . . .                                    | 17        |
| <b>12</b> | <b>Walkthrough thực nghiệm từng testcase</b>                         | <b>17</b> |
| 12.1      | Phương pháp phân tích một trace scheduler . . . . .                  | 17        |
| 12.2      | Walkthrough sched_prio: priority cao đến giữa quantum . . . . .      | 17        |
| 12.3      | Walkthrough sched_out_of_slot: quota và reset chu kỳ . . . . .       | 18        |
| 12.4      | Walkthrough sched_2_cpu: ownership trên nhiều CPU . . . . .          | 19        |
| 12.5      | Walkthrough os_mem_reuse: free list hoạt động . . . . .              | 19        |
| 12.6      | Walkthrough os_mem_roundtrip: dữ liệu qua user/kernel . . . . .      | 20        |
| 12.7      | Walkthrough os_cross_pid_protection: identity và ownership . . . . . | 20        |
| 12.8      | Walkthrough os_mm64_canonical: giải thích mọi con số . . . . .       | 20        |
| <b>13</b> | <b>Bài tính thực hành có lời giải</b>                                | <b>21</b> |
| 13.1      | Bài tính scheduler 1: priority và quantum . . . . .                  | 21        |
| 13.2      | Bài tính scheduler 2: weighted cycle . . . . .                       | 22        |
| 13.3      | Bài tính địa chỉ legacy . . . . .                                    | 22        |
| 13.4      | Bài tính internal fragmentation . . . . .                            | 22        |
| 13.5      | Bài tính MM64 . . . . .                                              | 22        |
| 13.6      | Bài mô phỏng swap từng bước . . . . .                                | 23        |
| <b>14</b> | <b>Các quyết định thiết kế và giới hạn cần nói thẳng</b>             | <b>23</b> |
| 14.1      | Các quyết định hợp lý . . . . .                                      | 23        |
| 14.2      | Các giới hạn kỹ thuật . . . . .                                      | 23        |
| 14.3      | Nếu được cải tiến . . . . .                                          | 24        |
| <b>15</b> | <b>Bộ câu hỏi vấn đáp kèm đáp án</b>                                 | <b>24</b> |
| 15.1      | Câu hỏi nền tảng . . . . .                                           | 24        |

|           |                                                        |           |
|-----------|--------------------------------------------------------|-----------|
| 15.2      | Câu hỏi scheduler . . . . .                            | 24        |
| 15.3      | Câu hỏi synchronization . . . . .                      | 25        |
| 15.4      | Câu hỏi memory . . . . .                               | 25        |
| 15.5      | Câu hỏi MM64 . . . . .                                 | 26        |
| 15.6      | Câu hỏi về kiểm thử và báo cáo . . . . .               | 26        |
| <b>16</b> | <b>Phiên vấn đáp mô phỏng hoàn chỉnh</b>               | <b>26</b> |
| 16.1      | Cách trả lời một câu vấn đáp kỹ thuật . . . . .        | 26        |
| 16.2      | Vòng 1: trình bày tổng quan trong hai phút . . . . .   | 27        |
| 16.3      | Vòng 2: đào sâu scheduler và synchronization . . . . . | 27        |
| 16.4      | Vòng 3: đào sâu memory và protection . . . . .         | 28        |
| 16.5      | Vòng 4: đào sâu MM64 . . . . .                         | 28        |
| 16.6      | Vòng 5: phản biện kiểm thử và chất lượng . . . . .     | 29        |
| 16.7      | Các câu hỏi bất thường gặp . . . . .                   | 29        |
| <b>17</b> | <b>Bài tập tự luyện</b>                                | <b>30</b> |
| 17.1      | Mức 1: đọc và dự đoán . . . . .                        | 30        |
| 17.2      | Mức 2: giải thích bằng source . . . . .                | 30        |
| 17.3      | Mức 3: thiết kế testcase . . . . .                     | 30        |
| 17.4      | Đáp án nhanh mức 1 . . . . .                           | 30        |
| 17.5      | Gợi ý lời giải mức 2 . . . . .                         | 31        |
| 17.6      | Gợi ý lời giải mức 3 . . . . .                         | 31        |
| <b>18</b> | <b>Cheat sheet trước khi trình bày</b>                 | <b>31</b> |
| <b>19</b> | <b>Kết luận học tập</b>                                | <b>32</b> |

# 1 Bức tranh tổng thể của hệ điều hành mô phỏng

## 1.1 Hệ điều hành thực sự làm gì?

Hệ điều hành đứng giữa chương trình người dùng và tài nguyên phần cứng. Trong bài tập này, phần cứng được đơn giản hóa thành các CPU thread, RAM và SWAP; chương trình người dùng được đơn giản hóa thành danh sách instruction. Dù vậy, ba trách nhiệm cốt lõi vẫn giống hệ điều hành thật:

- **Chia sẻ CPU:** quyết định process nào được chạy và chạy bao lâu.
- **Ảo hóa bộ nhớ:** mỗi process nhìn thấy address space riêng, dù tất cả cùng dùng RAM vật lý.
- **Kiểm soát đặc quyền:** user code không tự ý đọc/ghi tài nguyên vật lý; thao tác nhạy cảm phải đi qua kernel interface.

## 1.2 Kiến trúc end-to-end

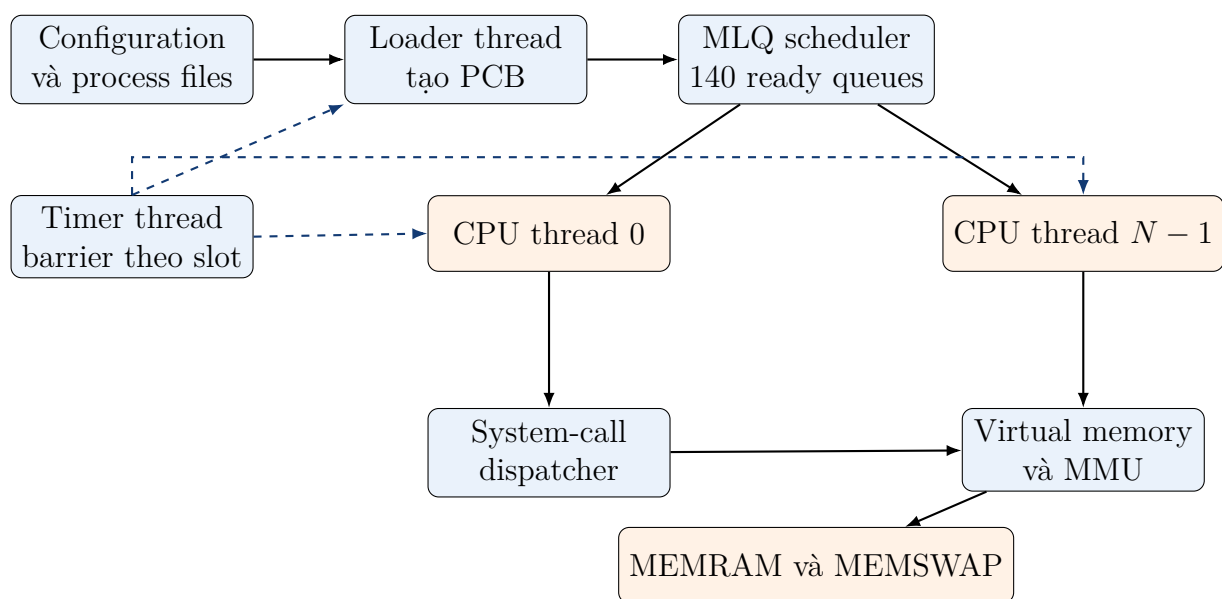


Figure 1: Luồng chính của simulator

Luồng chạy bắt đầu tại `src/os.c`. Hàm `main()` đọc configuration, tạo timer event, RAM/SWAP, scheduler, loader thread và các CPU thread. Loader đọc từng process description bằng `src/loader.c`, tạo PCB, chờ đến `start_time`, rồi gọi `add_proc()`. CPU gọi `get_proc()`, thực thi instruction qua `run()`, và trả process về scheduler khi hết quantum.

## 1.3 Phân biệt simulation và hệ điều hành thật

| Khía cạnh | Trong simulator | Trong OS thật |
|-----------|-----------------|---------------|
|-----------|-----------------|---------------|

|                 |                                     |                                              |
|-----------------|-------------------------------------|----------------------------------------------|
| CPU             | Một pthread cho mỗi CPU mô phỏng    | Core phần cứng thực thi machine instructions |
| Process code    | Mảng <code>struct inst_t</code>     | Text segment chứa mã máy                     |
| Context switch  | Đổi con trỏ PCB giữa các queue      | Lưu/phục hồi registers, stack, MMU state     |
| Timer interrupt | Condition variable/barrier          | Hardware timer interrupt                     |
| Page table      | Cấu trúc C trong heap của simulator | Cấu trúc mà MMU phần cứng đọc                |
| System call     | Gọi hàm C qua dispatcher            | Trap chuyển user mode sang kernel mode       |

**Điểm cần nhớ.** Simulator không tái tạo mọi chi tiết phần cứng. Mục tiêu là bảo toàn đúng *quan hệ khái niệm*: process không tự dùng tài nguyên vật lý, scheduler điều phối CPU, và kernel đồng bộ dữ liệu dùng chung.

## 2 Process, PCB, loader và CPU

### 2.1 Process và program khác nhau thế nào?

**Program** là nội dung tĩnh nằm trong file. **Process** là một instance đang chạy của program, có trạng thái riêng: PID, program counter, registers, priority và address space. Hai process có thể dùng cùng chương trình nhưng vẫn có PID và bộ nhớ độc lập.

### 2.2 PCB là trung tâm của trạng thái process

Trong repository, `struct pcb_t` nằm ở `include/common.h`. Các field quan trọng:

| Field          | Khái niệm                             | Vai trò                                                                      |
|----------------|---------------------------------------|------------------------------------------------------------------------------|
| pid            | Process identifier                    | Kernel dùng để định danh và lookup process.                                  |
| code, pc       | Text segment và program counter       | Instruction kế tiếp là <code>code-&gt;text[pc]</code> .                      |
| regs[]         | Registers mô phỏng                    | Lưu địa chỉ region hoặc dữ liệu đọc về.                                      |
| priority, prio | Priority cố định và priority khi chạy | Scheduler MLQ dùng <code>prio</code> ; số nhỏ hơn có ưu tiên cao hơn.        |
| krnl           | Kernel handle                         | Cho process truy cập kernel thông qua interface được kiểm soát.              |
| mm             | Address space                         | VMA, region table, page tables, FIFO victim list, kernel region và thống kê. |

## 2.3 Loader biến file thành PCB

Hàm `load()` trong `src/loader.c` thực hiện:

1. Dùng `calloc()` tạo PCB được zero-initialize.
2. Gán PID tăng dần từ `avail_pid`.
3. Đọc priority và số instruction ở đầu process file.
4. Chuyển chuỗi như `calc`, `alloc`, `syscall` thành `enum ins_opcode_t`.
5. Đọc argument của từng instruction vào `struct inst_t`.

Ví dụ process:

Listing 1: Process kiểm tra tái sử dụng region

```

1 20 4
2 alloc 100 1
3 free 1
4 alloc 50 2
5 read 2 49 3

```

Dòng đầu cho biết default priority là 20 và có bốn instruction. Lệnh đầu cấp 100 byte, dùng region/register ID 1. Sau khi free, lần cấp phát 50 byte vào region 2 phải tái sử dụng khoảng trống cũ.

## 2.4 CPU fetch–execute như thế nào?

Hàm `run()` trong `src/cpu.c` kiểm tra `pc`, fetch instruction, tăng `pc`, sau đó dispatch theo opcode. Có thể hình dung:

$$\text{instruction} \leftarrow \text{code}[\text{text}][\text{pc}], \quad \text{pc} \leftarrow \text{pc} + 1$$

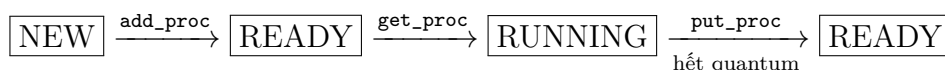
Listing 2: Mô hình rút gọn của CPU

```

1 if (proc->pc >= proc->code->size)
2     return 1;
3 ins = proc->code->text[proc->pc];
4 proc->pc++;
5 switch (ins.opcode) {
6 case CALC: return calc(proc);
7 case ALLOC: return liballoc(proc, ins.arg_0, ins.arg_1);
8 case SYSCALL: return libsyscall(proc, ins.arg_0, ins.arg_1,
9                               ins.arg_2, ins.arg_3);
10 }

```

## 2.5 Vòng đời PCB







Khi process hoàn tất, CPU gọi `finish_proc()`, in page table, gọi `destroy_mm()`, rồi giải phóng code segment, legacy page table và PCB.

## 2.6 Câu hỏi tự kiểm tra

**Câu hỏi:** Tại sao loader dùng `calloc()` thay vì chỉ dùng `malloc()` cho PCB?

**Trả lời ngắn:** Vì nhiều field, đặc biệt registers và con trỏ, cần khởi tạo bằng 0. Nếu để giá trị rác, CPU hoặc memory subsystem có thể hiểu nhầm một region chưa được cấp phát là hợp lệ.

**Câu hỏi:** Tại sao `pc` được tăng trước khi instruction hoàn tất?

**Trả lời ngắn:** Đây là lựa chọn mô phỏng fetch-execute. Nếu instruction lỗi, simulator hiện tại vẫn xem nó đã được tiêu thụ. Một thiết kế khác có thể chỉ commit `pc` sau khi thành công, nhưng semantics sẽ khác và cần được quy định rõ.

## 3 CPU Scheduling và Multi-Level Queue

### 3.1 Bài toán scheduler

Khi số process sẵn sàng lớn hơn số CPU, scheduler phải chọn process nào được chạy. Một chính sách tốt thường cân bằng:

- **Response time:** process quan trọng được phản hồi sớm.
- **Throughput:** hoàn thành nhiều công việc.
- **Fairness:** process ưu tiên thấp không bị bỏ quên mãi.
- **Overhead:** quyết định scheduling không quá tốn kém.

### 3.2 MLQ trong repository

Scheduler định nghĩa `MAX_PRIO = 140`, nên có 140 ready queue:

$$Q_0, Q_1, \dots, Q_{139}$$

Priority nhỏ hơn nghĩa là ưu tiên cao hơn. Mỗi queue dùng FIFO để bảo toàn thứ tự đến. Hàm `dequeue()` trong `src/queue.c` luôn lấy phần tử đầu, còn việc so sánh priority thuộc về tầng MLQ.

### 3.3 Weighted slot

Mỗi mức priority nhận số slot:

$$slot[p] = MAX\_PRIO - p$$

Do đó  $Q_0$  có 140 slot trong một chu kỳ, còn  $Q_{139}$  có 1 slot. `get_mlq_proc()` quét từ priority cao đến thấp, chỉ chọn queue vừa không rỗng vừa còn slot. Khi không chọn được process nào, toàn bộ slot được reset.

Listing 3: Ý tưởng chọn process của MLQ

```

1 for (prio = 0; prio < MAX_PRIO; prio++) {
2     if (!empty(&mlq_ready_queue[prio]) && slot[prio] > 0) {
3         proc = dequeue(&mlq_ready_queue[prio]);
4         slot[prio]--;
5         break;
6     }
7 }
8 if (proc == NULL)
9     reset_all_slots_and_try_again();

```

### 3.4 Priority và quantum là hai khái niệm khác nhau

**Priority** trả lời “ai được chọn tiếp theo?”. **Quantum** trả lời “process được chọn chạy liên tục bao nhiêu instruction trước khi phải trả CPU?”. Trong `cpu_routine()`, biến `time_left` bắt đầu bằng `time_slot`; mỗi lần gọi `run(proc)` thì giảm một.

Process ưu tiên cao mới đến không lập tức cướp CPU giữa quantum hiện tại. Nó được chọn ở biên dispatch tiếp theo. Đây là preemption theo quantum, không phải preemption tức thời theo từng thời điểm đến.

### 3.5 Running list giải quyết vấn đề gì?

Khi process được lấy khỏi ready queue, nó được thêm vào `running_list`. Danh sách này có hai vai trò:

- Theo dõi process đang được CPU giữ, tránh dispatch cùng PCB cho hai CPU.
- Cho kernel lookup PCB bằng PID khi process thực hiện syscall.

### 3.6 Bất biến scheduler

1. Một PCB không đồng thời xuất hiện trong ready queue và running list.
2. Một lần `get_proc()` thành công phải loại PCB khỏi ready queue và thêm nó vào running list.
3. `put_proc()` phải xóa PCB khỏi running list trước khi enqueue lại.
4. Mọi thao tác làm thay đổi hoặc duyệt queue dùng chung phải được bảo vệ bởi `queue_lock`.
5. Trong mỗi priority queue, thứ tự dispatch là FIFO.

### 3.7 Độ phức tạp và trade-off

| Thao tác      | Độ phức tạp             | Nhận xét                                                                         |
|---------------|-------------------------|----------------------------------------------------------------------------------|
| Quét chọn MLQ | $O(MAX\_PRIO)$          | 140 là hằng số nhỏ trong bài tập.                                                |
| Enqueue       | $O(1)$                  | Thêm vào cuối mảng queue.                                                        |
| Dequeue/purge | $O(n)$                  | Phải dịch phần tử trong mảng. Có thể tối ưu bằng ring buffer hoặc linked list.   |
| PID lookup    | $O(\text{tổng số PCB})$ | Đơn giản, phù hợp simulator; OS thật thường dùng hash table hoặc PID radix tree. |

**Điểm cần nhớ.** Weighted slot giảm nguy cơ starvation nhưng không tạo fairness tuyệt đối. Nếu luồng priority cao luôn đông, luồng thấp vẫn có thể chờ lâu; thời gian chờ phụ thuộc chu kỳ reset slot và thời lượng quantum.

### 3.8 Đọc testcase scheduler

Với input/sched\_prio:

```

1 2 1 2
2 0 s1 1
3 1 s0 0

```

Quantum là 2, có một CPU và hai process. PID 1/prio 1 đến lúc 0, PID 2/prio 0 đến lúc 1. PID 1 vẫn hoàn thành quantum đang chạy; đến biên dispatch, PID 2 được chọn do priority 0 cao hơn.

**Câu hỏi:** Tại sao không đặt logic priority trực tiếp trong `dequeue()`?

**Trả lời ngắn:** Vì mỗi MLQ bucket đã đại diện cho một priority. `dequeue()` chỉ cần giữ FIFO nội bộ; `get_mlq_proc()` quyết định bucket nào thắng. Tách hai trách nhiệm giúp queue có semantics rõ ràng.

## 4 Concurrency, timer và synchronization

### 4.1 Tại sao simulator cần synchronization?

Loader và nhiều CPU là các pthread chạy đồng thời. Chúng cùng truy cập queue, RAM free-frame list, page tables và trạng thái timer. Một biểu thức C tưởng như đơn giản có thể gồm nhiều bước máy:

$$q.size \leftarrow q.size + 1$$

Nếu hai thread đọc cùng giá trị cũ rồi cùng ghi giá trị mới, một lần enqueue có thể bị mất. Đây là **race condition**.

## 4.2 Mutex và condition variable

| Cơ chế             | Mục tiêu                                              | Ví dụ trong repository                                                        |
|--------------------|-------------------------------------------------------|-------------------------------------------------------------------------------|
| Mutex              | Chỉ một thread vào critical section tại một thời điểm | <code>queue_lock</code> , <code>mmvm_lock</code> , <code>memphy_lock</code> . |
| Condition variable | Cho thread ngủ đến khi điều kiện trạng thái thay đổi  | <code>event_cond</code> , <code>timer_cond</code> .                           |
| Atomic variable    | Đọc/ghi một trạng thái nhỏ mà không data race         | <code>atomic_int done</code> .                                                |

## 4.3 Timer barrier theo time slot

Timer không chỉ “đếm giờ”. Nó tạo barrier để mỗi thiết bị mô phỏng làm đúng một bước trong một slot:

1. CPU/loader hoàn thành phần việc hiện tại rồi gọi `next_slot()`.
2. `next_slot()` đặt cờ `done = 1`, báo `event_cond`, rồi ngủ chờ `timer_cond`.
3. Timer chờ tất cả event báo hoàn tất.
4. Timer tăng `_time`, reset cờ và đánh thức các event.

Mô hình này làm cho execution trace dễ phân tích hơn và ngăn một CPU thread chạy nhanh hơn vô hạn chỉ vì host scheduler ưu ái nó.

## 4.4 Phạm vi lock và deadlock

Lock nên bao quanh đúng critical section. Giữ lock quá rộng làm giảm song song; giữ quá hẹp làm bất biến bị phá vỡ. Ví dụ, thao tác chuyển PCB từ running list về ready queue phải là một critical section duy nhất:

```

1 pthread_mutex_lock(&queue_lock);
2 purgequeue(&running_list, proc);
3 enqueue(&mlq_ready_queue[proc->prio], proc);
4 pthread_mutex_unlock(&queue_lock);

```

**Deadlock** xảy ra khi các thread chờ lock theo vòng tròn. Repository hiện tránh phần lớn nguy cơ này bằng các lock tách theo subsystem và không giữ `queue_lock` trong lúc thực thi memory operation dài.

**Câu hỏi:** Điều gì xảy ra nếu bỏ `queue_lock`?

**Trả lời ngắn:** Hai CPU có thể lấy cùng PCB, kích thước queue bị cập nhật sai, hoặc `purgequeue()` dịch mảng trong khi thread khác đang duyệt. Kết quả có thể không tái hiện ổn định, từ sai lệch đến crash.

## 5 System call và ranh giới user/kernel

### 5.1 Tại sao cần system call?

User process không nên cầm con trỏ kernel hoặc tự đọc RAM vật lý. System call là cổng kiểm soát: caller cung cấp số syscall và arguments; kernel dispatcher chọn handler, xác định caller và kiểm tra quyền trước khi thao tác.

### 5.2 Đường đi của một memory operation

ALLOC instruction  $\rightarrow$  liballoc()  $\rightarrow$  \_syscall(...,17,...)  
 $\rightarrow$  \_\_sys\_mmap()  $\rightarrow$  \_\_alloc()

src/syscall.tbl định nghĩa:

```
1 0 listsyscall sys_listsyscall
2 17 mmap sys_mmap
3 440 xxx sys_xxxhandler
```

Script sinh src/syscalltbl.lst; src/syscall.c include danh sách này nhiều lần với macro khác nhau để tạo prototype, bảng tên và switch dispatcher. Đây là kỹ thuật X-macro: một nguồn dữ liệu sinh nhiều cấu trúc nhất quán.

### 5.3 Unified memory syscall

Syscall 17 dùng regs->a1 làm operation selector. Các operation gồm map, grow VMA, swap, physical I/O, alloc/free/read/write, kernel allocation, cache pool và copy user/kernel.

| Operation            | Handler phía kernel | Ý nghĩa                                  |
|----------------------|---------------------|------------------------------------------|
| SYSTEM_ALLOC_OP      | __alloc()           | Cấp user region.                         |
| SYSTEM_FREE_OP       | __free()            | Trả region cho free list.                |
| SYSTEM_IO_READ/WRITE | MEMPHY_read/write() | Physical I/O sau khi kiểm tra ownership. |
| SYSTEM_SWP_OP        | __mm_swap_page()    | Copy frame RAM sang SWAP.                |
| SYSTEM_KMALLOC_OP    | __kmalloc()         | Cấp kernel region có frame liên tiếp.    |

### 5.4 Kernel lookup PCB bằng PID

Dispatcher nhận pid, không nhận trực tiếp PCB từ user interface. \_\_sys\_mmap() gọi find\_proc\_by\_pid(pid) để lấy caller thực từ các queue do kernel quản lý. Điều này mô phỏng nguyên tắc:

Không tin con trỏ/identity do user cung cấp

## 5.5 Bảo vệ raw physical access

Trước khi `SYSMEM_IO_READ` hoặc `SYSMEM_IO_WRITE`, kernel gọi `caller_owns_phyaddr()`. Trong MM64, hàm `paging64_owns_fpn()` kiểm tra frame có xuất hiện trong page table của caller hay không. Nếu PID 2 cố đọc frame thuộc PID 1, kernel trả lỗi và log:

```
1 Kernel denied PID 2 physical read at 0x0
2 Kernel denied PID 2 physical write at 0x0
```

**Điểm cần nhớ.** Address translation và access control là hai việc liên quan nhưng không đồng nhất. Dịch được một địa chỉ không có nghĩa caller được phép truy cập nó.

## 5.6 Ưu và nhược điểm của unified syscall

**Ưu điểm:**

- Chỉ một entry point cần nối vào syscall table.
- Dễ gom kiểm tra caller, logging và policy.
- Mở rộng operation bằng selector mà không tiêu tốn thêm syscall number.

**Nhược điểm:**

- Handler switch lớn và nhiều argument có semantics khác nhau.
- Type safety kém hơn các syscall chuyên biệt.
- Nếu validation không nhất quán, một operation có thể trở thành đường vòng vượt qua protection.

# 6 Nền tảng quản lý bộ nhớ

## 6.1 Ba loại địa chỉ cần phân biệt

- **Virtual address:** địa chỉ process nhìn thấy.
- **Physical address:** vị trí byte thật trong MEMRAM.
- **Page/frame number:** chỉ số block có kích thước cố định.

Với page size  $P$ :

$$pgn = \left\lfloor \frac{vaddr}{P} \right\rfloor, \quad offset = vaddr \bmod P$$

$$paddr = fpn \times P + offset$$

Trong đường memory hiện tại, `PAGING_PAGESZ = 256` byte cho cơ chế allocation và PTE. MM64 bổ sung phân rã địa chỉ 57-bit với page size logic 4096 byte khi khảo sát page-table hierarchy. Đây là hai lớp mô phỏng cần phân biệt khi giải thích.

## 6.2 VMA, region và symbol table

**VMA** mô tả một miền địa chỉ ảo liên tục, có `vm_start`, `vm_end`, `sbrk` và free-region list. **Region** là allocation được định danh bằng `rgid`. `symrgtbl[rgid]` lưu khoảng `[rg_start, rg_end)`.

Quy ước half-open interval rất quan trọng:

$$[start, end) \Rightarrow size = end - start$$

Byte cuối hợp lệ ở `end - 1`, còn `end` đã nằm ngoài region.

## 6.3 ALLOC

`__alloc()` trước hết thử tìm một vùng đủ lớn trong free-region list. Nếu không có, nó tăng VMA:

1. Lưu `old_sbrk`.
2. Gọi `inc_vma_limit()` với size yêu cầu.
3. Align phần map vật lý theo page size.
4. Map các frame vào page table.
5. Đặt region chính xác từ `old_sbrk` đến `old_sbrk + size`.
6. Phần dư do alignment được thêm vào free-region list.

## 6.4 FREE và tái sử dụng

`__free()` không nhất thiết unmap page ngay. Nó xóa entry trong symbol table và đưa khoảng region vào free list. Allocation kế tiếp có thể lấy vùng này trước khi mở rộng `sbrk`. Test `os_mem_reuse` chứng minh:

```
1 MEM alloc PID 1 region 1 [0,100) size 100
2 MEM free PID 1 region 1 [0,100)
3 MEM alloc PID 1 region 2 [0,50) size 50
```

## 6.5 READ/WRITE và kiểm tra biên

Một access theo region cần thỏa:

$$0 \leq offset < rg\_end - rg\_start$$

Virtual address được tạo bằng `rg_start + offset`, sau đó page table cho biết frame. Cuối cùng physical address được kernel I/O handler đọc/ghi sau khi kiểm tra ownership.

## 6.6 Physical memory và free-frame list

struct memphy\_struct chứa:

- **storage**: mảng byte mô phỏng thiết bị.
- **maxsz**: dung lượng hiện thực được cấp.
- **free\_fp\_list**: danh sách frame chưa dùng.
- **rdmflg**: random-access hay sequential mode.

MEMPHY\_get\_freefp() lấy frame đầu free list dưới memphy\_lock; MEMPHY\_put\_freefp() trả frame về. MEMPHY\_get\_contiguous\_fps() quét tìm chuỗi frame liên tiếp cho kernel allocation.

## 6.7 Paging so với continuous allocation

|            | Paging                                                                             | Continuous allocation                                                                  |
|------------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Ưu điểm    | Không cần một khối vật lý lớn; giảm external fragmentation; hỗ trợ swap theo page. | Translation đơn giản; phù hợp DMA/kernel object cần liên tiếp; ít page-table overhead. |
| Nhược điểm | Page-table overhead; internal fragmentation; page fault và translation cost.       | External fragmentation; khó tìm vùng lớn; compaction tốn kém.                          |

## 7 Page table, PTE và swapping

### 7.1 PTE biểu diễn trạng thái gì?

Page-table entry không chỉ chứa frame number. Trong `include/mm.h`, PTE có các bit/mảng bit cho:

- **PRESENT**: page đang ở RAM.
- **SWAPPED**: page đang ở SWAP.
- **DIRTY**: page đã bị ghi thay đổi.
- **FPN** hoặc cặp swap type/swap offset.

Một page hợp lệ thường ở một trong hai trạng thái:

$$PRESENT = 1, SWAPPED = 0 \quad \text{hoặc} \quad PRESENT = 0, SWAPPED = 1$$



## 7.2 Khi page không ở RAM

`pg_getpage()` xử lý page fault mô phỏng:

1. Đọc PTE của target page.
2. Nếu không present, chọn victim bằng FIFO.
3. Lấy một frame trống trong SWAP.
4. Copy victim RAM frame sang SWAP qua syscall.
5. Cập nhật victim PTE thành swapped.
6. Nếu target từng ở SWAP, copy nó về victim frame.
7. Cập nhật target PTE thành present và đưa target vào FIFO list.

## 7.3 FIFO replacement

FIFO chọn page vào RAM sớm nhất làm victim. Ưu điểm là đơn giản và ít metadata; nhược điểm là không quan tâm page có đang được dùng thường xuyên hay không. FIFO còn có thể gặp **Belady's anomaly**: tăng số frame đôi khi lại làm tăng số page fault.

## 7.4 Bất biến bộ nhớ

1. Một frame RAM không được đồng thời thuộc hai process nếu không có cơ chế shared memory rõ ràng.
2. Mọi region hợp lệ có `start < end`; region trống dùng `start = end = 0`.
3. Access phải nằm trong region và caller phải sở hữu frame vật lý.
4. Frame lấy khỏi free list phải được map hoặc rollback nếu allocation lỗi.
5. Khi process kết thúc, các user frame và cấu trúc page table phải được giải phóng.

**Câu hỏi:** *Segmentation và paging kết hợp để làm gì?*

**Trả lời ngắn:** Segmentation/VMA cung cấp ý nghĩa logic và protection theo miền như code, heap hoặc region; paging cho phép map các miền đó lên frame rồi rạc, giảm yêu cầu liên tục vật lý. Kết hợp hai cơ chế vừa giữ tổ chức logic vừa tận dụng RAM linh hoạt.

# 8 Kernel memory, cache pool và copy user/kernel

## 8.1 User memory và kernel memory

User memory thuộc address space process và phải được kiểm tra khi truy cập. Kernel memory phục vụ dữ liệu nội bộ có đặc quyền. Repository dùng dải bắt đầu từ `PAGING_KMEM_BASE` để phân biệt kernel address.

## 8.2 KMALLOC

`__kmalloc()` tìm các physical frame liên tiếp và ghi metadata vào `ksymrgtbl`, `kphy_start` và `kphy_npages`. So với user `ALLOC`, yêu cầu contiguous frame làm allocation khó hơn và dễ thất bại do fragmentation.

## 8.3 Kernel cache pool

Cache pool mô phỏng ý tưởng slab allocator: tạo trước pool chứa nhiều object cùng kích thước/alignment, sau đó cấp object nhanh mà không lặp lại toàn bộ quy trình tìm frame.

Thông tin chính gồm `size`, `align`, `used`, `capacity`, `physical_storage` và `virtual storage`.

## 8.4 Copy qua ranh giới protection

`copy_from_user` và `copy_to_user` thể hiện nguyên tắc kernel không nên tùy tiện dereference user pointer:

- `copy_from_user`: đọc region user đã kiểm tra, ghi sang kernel.
- `copy_to_user`: đọc kernel region hợp lệ, ghi về user region.

Parser hỗ trợ cả cú pháp đặc tả và dạng mở rộng. Với dạng đặc tả, `copy_from_user source destination` mặc định `offset = 0`, `size = 1`; `copy_to_user source destination offset` mặc định `size = 1`. Dạng mở rộng của cả hai lệnh nhận thêm `offset size` để copy nhiều byte.

Test `os_mem_roundtrip` ghi byte 65 vào user region, copy sang kernel, copy về một user region khác rồi đọc lại:

```
1 MEM alloc PID 1 region 1 [0,16) size 16
2 MEM alloc PID 1 region 3 [16,32) size 16
3 MEM read PID 1 region 3 offset 0 value 65
```

## 9 Multi-Level Paging 64-bit

### 9.1 Vì sao cần nhiều cấp?

Một page table phẳng cho address space lớn sẽ có số entry khổng lồ, kể cả khi process chỉ dùng vài vùng nhỏ. Hierarchical paging chỉ cấp bảng con cho nhánh địa chỉ thực sự được dùng, gọi là **sparse allocation**.

### 9.2 Phân rã địa chỉ 57-bit

Với page size 4 KiB, offset cần 12 bit. Năm cấp, mỗi cấp dùng 9 bit:

$$\underbrace{PGD}_9 \mid \underbrace{P4D}_9 \mid \underbrace{PUD}_9 \mid \underbrace{PMD}_9 \mid \underbrace{PT}_9 \mid \underbrace{offset}_{12} = 57 \text{ bit}$$

| Thành phần | Bit   | Cách lấy               |
|------------|-------|------------------------|
| Offset     | 0–11  | $addr \& (2^{12} - 1)$ |
| PT         | 12–20 | $(addr \gg 12) \& 511$ |
| PMD        | 21–29 | $(addr \gg 21) \& 511$ |
| PUD        | 30–38 | $(addr \gg 30) \& 511$ |
| P4D        | 39–47 | $(addr \gg 39) \& 511$ |
| PGD        | 48–56 | $(addr \gg 48) \& 511$ |

### 9.3 Canonical address

Với virtual address width 57 bit trên kiểu 64 bit, bit 56 là sign bit. Các bit 57–63 phải là sign extension:

$$\begin{aligned} bit_{56} = 0 &\Rightarrow bits_{63:57} = 0 \\ bit_{56} = 1 &\Rightarrow bits_{63:57} = 1111111_2 \end{aligned}$$

`paging64_is_canonical()` kiểm tra điều kiện này. Address ở “lỗ” non-canonical phải bị từ chối thay vì alias sang một nhánh page table khác.

### 9.4 Page-table walk và sparse allocation

`walk_pte()` lần lượt:

$$PGD \rightarrow P4D \rightarrow PUD \rightarrow PMD \rightarrow PT \rightarrow PTE$$

Khi `create = 1`, bảng con chưa tồn tại được cấp bằng `calloc(512, sizeof(addr_t))`. Khi chỉ đọc với `create = 0`, nhánh thiếu dẫn đến lỗi, không tự tạo page table.

### 9.5 Thống kê

`mm_struct` theo dõi:

- `pgtable_walks`: số lần bắt đầu walk.
- `pgtable_accesses`: số table entry đã truy cập.
- `pgtable_bytes`: tổng byte bảng đã cấp.

Một bảng 512 entry, mỗi entry 8 byte:

$$512 \times 8 = 4096 \text{ byte}$$

Nhánh địa chỉ đầu tiên thường cần PGD và bốn bảng con. Các địa chỉ chia sẻ prefix có thể dùng lại nhiều bảng, nên sparse allocation tiết kiệm đáng kể so với cấp toàn bộ cây.

## 9.6 Lợi ích và chi phí của N-level paging

**Lợi ích:** hỗ trợ address space lớn, chỉ cấp nhánh cần dùng, phù hợp process thưa địa chỉ.  
**Chi phí:** nhiều memory access cho một translation, logic phức tạp, và page-table page vẫn có overhead. OS thật giảm chi phí walk bằng TLB.

**Câu hỏi:** Tại sao test cần cả *low canonical*, *high canonical* và *non-canonical*?

**Trả lời ngắn:** Chỉ test địa chỉ thấp không chứng minh sign-extension đúng. High canonical kiểm tra nhánh có bit 56 bằng 1 được chấp nhận; non-canonical kiểm tra khoảng lỗ bị từ chối.

## 10 Đọc source code theo luồng thay vì theo tên file

### 10.1 Luồng khởi động

1. `src/os.c`: đọc config, tạo tài nguyên và thread.
2. `src/timer.c`: dựng barrier time slot.
3. `src/sched.c`: khởi tạo 140 queue và slot.
4. `src/loader.c`: tạo PCB, parse instruction.
5. Loader gọi `add_proc()`, CPU bắt đầu `get_proc()`.

### 10.2 Luồng một instruction ALLOC

1. CPU fetch ALLOC trong `run()`.
2. `liballoc()` đóng gói `sc_regs`.
3. `mem_syscall()` gọi syscall số 17 kèm PID.
4. Dispatcher gọi `__sys_mmap()`.
5. Kernel lookup caller bằng PID.
6. `__alloc()` tìm free region hoặc tăng VMA.
7. `inc_vma_limit()` map thêm page/frame nếu cần.
8. Địa chỉ trả qua register syscall rồi được lưu vào `proc->regs[rgid]`.

### 10.3 Luồng một instruction READ

1. Kiểm tra region ID và offset.
2. Tính virtual address.
3. Lấy PTE; nếu page không present thì swap/page-in.
4. Tính physical address từ FPN và offset.

5. Gọi syscall physical read.
6. Kernel xác nhận caller sở hữu frame.
7. Đọc byte từ MEMRAM và trả về register.

## 10.4 Luồng process hết quantum và kết thúc

Hết quantum: CPU gọi `put_proc()`, scheduler chuyển PCB từ running list về MLQ bucket tương ứng. Khi `pc == code->size`, CPU gọi `finish_proc()`, giải phóng memory state và PCB, sau đó lấy process khác.

## 11 Kiểm thử có ý nghĩa

### 11.1 Ba tầng kiểm thử

1. **Build**: source compile và link thành binary.
2. **Execution**: testcase chạy đến cuối, không crash/deadlock.
3. **Semantic assertion**: output chứa bằng chứng cho invariant cần kiểm tra.

Việc “chạy không crash” chưa chứng minh đúng. Ví dụ, cross-PID read có thể chạy đến cuối nhưng vẫn là lỗi bảo mật nếu kernel không từ chối.

### 11.2 Traceability từ yêu cầu đến test

| Yêu cầu/bất biến              | Test tiêu biểu                       | Bằng chứng                                            |
|-------------------------------|--------------------------------------|-------------------------------------------------------|
| Priority và quantum           | <code>sched_prio</code>              | Process priority cao được chọn ở biên quantum.        |
| Đa CPU không trùng PCB        | <code>sched_2_cpu</code>             | Hai CPU dispatch PID khác nhau tại cùng thời điểm.    |
| FREE cho phép reuse           | <code>os_mem_reuse</code>            | Region 2 bắt đầu tại địa chỉ 0 sau khi region 1 free. |
| Copy user-kernel đúng dữ liệu | <code>os_mem_roundtrip</code>        | Byte 65 đi vòng và đọc lại không đổi.                 |
| Cross-PID protection          | <code>os_cross_pid_protection</code> | Kernel denied read và write của PID 2.                |
| Canonical MM64                | <code>os_mm64_canonical</code>       | Low/high accepted, non-canonical rejected.            |
| Syscall table đúng            | <code>os_syscall_list</code>         | Có syscall 0, 17 và 440.                              |

### 11.3 Cách đọc log

Khi đọc log, luôn hỏi:

1. Dòng này do subsystem nào in?
2. Nó xảy ra trước hay sau một boundary quan trọng như quantum, syscall, page fault?
3. Nó chứng minh invariant nào?
4. Có output nào cũng hợp lệ do concurrency làm thay đổi thứ tự không?

## 11.4 Lệnh kiểm chứng khi học

```
1 make clean
2 make all
3 make test
4
5 ./os sched_prio
6 ./os os_mem_reuse
7 ./os os_cross_pid_protection
8 ./os os_mm64_canonical
```

**Điểm cần nhớ.** Output concurrent có thể khác thứ tự ở các dòng độc lập. Assertion tốt nên kiểm tra quan hệ ngữ nghĩa hoặc mốc bắt buộc, không đòi toàn bộ output giống tuyệt đối nếu đặc tả không yêu cầu deterministic ordering.

## 12 Walkthrough thực nghiệm từng testcase

### 12.1 Phương pháp phân tích một trace scheduler

Để không bị rối bởi hàng chục dòng `Time slot`, ta lập một bảng trạng thái chỉ tại các mốc có sự kiện:

1. Ghi lại process mới được loader thêm vào ready queue.
2. Ghi process đang chạy trên từng CPU và `time_left`.
3. Chỉ cập nhật quota MLQ khi `get_proc()` thực sự lấy một PCB.
4. Phân biệt “hết quantum” với “process hoàn tất”.
5. Sau mỗi dispatch, kiểm tra PCB vừa chọn không còn ở ready queue.

Một dòng `Dispatched process` nghĩa là scheduler vừa quyết định; một dòng `Put process` nghĩa là process cũ hết quantum nhưng chưa kết thúc. Nếu process kết thúc đúng sau instruction cuối, CPU in `Processed ... has finished` thay vì put nó trở lại ready queue.

### 12.2 Walkthrough sched\_prio: priority cao đến giữa quantum

Configuration:

```

1 2 1 2
2 0 s1 1
3 1 s0 0

```

Quantum bằng 2, một CPU. PID 1 có priority 1 và bảy instruction; PID 2 có priority 0 và mười lăm instruction. Initial quota:

$$slot[0] = 140, \quad slot[1] = 139$$

| Mốc       | Ready/running                     | Quyết định               | Giải thích                                                     |
|-----------|-----------------------------------|--------------------------|----------------------------------------------------------------|
| Slot 0–1  | PID 1 vào $Q_1$                   | CPU dispatch PID 1       | Tại thời điểm chọn chỉ PID 1 sẵn sàng; $slot[1]$ giảm còn 138. |
| Slot 1    | PID 2 vào $Q_0$ ; PID 1 đang chạy | Không preempt tức thời   | PID 1 vẫn còn quantum hiện tại.                                |
| Slot 3    | PID 1 được put vào $Q_1$          | Dispatch PID 2           | $Q_0$ có priority cao hơn $Q_1$ ; $slot[0]$ giảm.              |
| Slot 3–18 | PID 2 liên tục quay lại $Q_0$     | PID 2 tiếp tục được chọn | Priority 0 còn rất nhiều weighted slot nên PID 1 phải chờ.     |
| Slot 18   | PID 2 hoàn tất                    | Dispatch PID 1           | $Q_0$ rỗng nên PID 1 tiếp tục.                                 |

Điểm quan trọng nhất: priority cao không cướp CPU ở giữa quantum. Tuy nhiên, sau khi có cơ hội scheduling, process priority thấp có thể bị trì hoãn lâu vì weighted quota của priority cao rất lớn.

**Câu hỏi:** Nếu muốn priority 0 preempt ngay khi vừa đến thì cần thay đổi gì?

**Trả lời ngắn:** Loader phải phát tín hiệu reschedule hoặc CPU phải kiểm tra ready queue sau mỗi instruction. Thiết kế đó tăng response time cho priority cao nhưng làm tăng scheduling overhead và cần đồng bộ phức tạp hơn.

## 12.3 Walkthrough sched\_out\_of\_slot: quota và reset chu kỳ

Configuration dùng ba priority thấp nhất theo nghĩa quota:

$$slot[137] = 3, \quad slot[138] = 2, \quad slot[139] = 1$$

PID 1/prio 139 được dispatch đầu tiên vì đến trước, tiêu thụ quota duy nhất của  $Q_{139}$ . Khi PID 2/prio 138 và PID 3/prio 137 xuất hiện, scheduler ưu tiên PID 3 ba lần, rồi PID 2 hai lần:

| Thứ tự dispatch | PID | Quota còn lại sau chọn | Nhận xét |
|-----------------|-----|------------------------|----------|
|-----------------|-----|------------------------|----------|

|     |                                          |                    |                                                              |
|-----|------------------------------------------|--------------------|--------------------------------------------------------------|
| 1   | PID 1, $p = 139$                         | $slot[139] = 0$    | Đến trước khi process khác sẵn sàng.                         |
| 2–4 | PID 3, $p = 137$                         | 2, 1, 0            | Priority cao nhất trong ready set, dùng đủ ba weighted slot. |
| 5–6 | PID 2, $p = 138$                         | 1, 0               | Sau khi $Q_{137}$ hết quota, $Q_{138}$ được phục vụ.         |
| 7   | Không queue nào vừa có PCB vừa còn quota | Reset toàn bộ slot | $Q_{139}$ có PID 1 nhưng quota bằng 0; reset mở chu kỳ mới.  |

Sau reset, PID 3 lại được chọn trước vì priority 137. Khi PID 3 và PID 2 hoàn tất, PID 1 cuối cùng được chạy tiếp. Test này chứng minh hai ý: scheduler tôn trọng weighted quota và không làm mất process khi queue tạm hết slot.

## 12.4 Walkthrough sched\_2\_cpu: ownership trên nhiều CPU

Hai CPU có thể dispatch đồng thời, nhưng mỗi PCB chỉ thuộc một CPU:

| Slot | CPU 0                          | CPU 1                      | Điều cần kiểm tra                                                        |
|------|--------------------------------|----------------------------|--------------------------------------------------------------------------|
| 0–1  | Dispatch PID 1                 | Chưa có PCB                | PID 1 nằm trong running list.                                            |
| 2    | Put PID 1, dispatch PID 3      | Dispatch PID 2             | PID 2 và PID 3 khác nhau; không CPU nào lấy PID 1 trước khi nó được put. |
| 4    | PID 3 đang chạy                | Put PID 2, dispatch PID 1  | PID 1 đã rời ready queue và thuộc CPU 1.                                 |
| 15   | PID 3 hoàn tất; dispatch PID 1 | PID 2 hoàn tất, CPU 1 dừng | PID 1 chỉ được một CPU giữ.                                              |

Không nên kiểm tra cứng CPU nào phải nhận PID nào, vì host thread scheduling có thể đổi thứ tự hợp lệ. Invariant thực sự là tại cùng thời điểm, hai CPU không được giữ cùng PCB.

## 12.5 Walkthrough os\_mem\_reuse: free list hoạt động

Process thực hiện:

```
alloc 100 1 → free 1 → alloc 50 2 → read 2 49 3
```

1. Allocation đầu tiên không tìm thấy free region, nên mở rộng VMA và ghi `symrgtbl[1] = [0, 100)`.
2. `free 1` tạo node `[0, 100)`, thêm vào `vm_freerg_list`, rồi xóa region/register 1.
3. Allocation 50 byte tìm được vùng cũ đủ lớn, nhận `[0, 50)`.



- Read offset 49 truy cập byte cuối hợp lệ của region mới; giá trị bằng 0 vì MEMRAM được zero-initialize.

Đây là bằng chứng cho *logical region reuse*, không nhất thiết chứng minh frame vật lý đã được unmap rồi cấp lại. Page có thể vẫn thuộc address space của PID 1.

## 12.6 Walkthrough os\_mem\_roundtrip: dữ liệu qua user/kernel

Chuỗi dữ liệu cần theo dõi:



| Instruction               | Trạng thái sau lệnh           | Ý nghĩa                                          |
|---------------------------|-------------------------------|--------------------------------------------------|
| alloc 16 1                | User region 1 là [0, 16)      | Tạo nguồn dữ liệu.                               |
| write 65 1 0              | Byte đầu region 1 bằng 65     | Kiểm tra user write.                             |
| copy_from_user<br>1 2 0 1 | Kernel region 2 nhận một byte | Kernel đọc từ user qua interface được kiểm soát. |
| alloc 16 3                | User region 3 là [16, 32)     | Tạo đích user mới.                               |
| copy_to_user<br>2 3 0 1   | Byte đầu region 3 bằng 65     | Kernel copy dữ liệu về user.                     |
| read 3 0 4                | Register 4 nhận 65            | Chứng minh roundtrip bảo toàn dữ liệu.           |

## 12.7 Walkthrough os\_cross\_pid\_protection: identity và ownership

PID 1 cấp region [0, 100), dẫn đến frame vật lý đầu tiên thuộc address space PID 1. PID 2 sau đó gọi trực tiếp syscall 17 với operation physical read và write tại địa chỉ 0.

- Dispatcher nhận PID 2, không dùng PCB do user truyền.
- find\_proc\_by\_pid(2) trả PCB thật của intruder.
- caller\_owns\_phyaddr() kiểm tra frame 0 trong page table PID 2.
- Frame không thuộc PID 2 nên kernel từ chối cả read và write.

Nếu kernel chỉ kiểm tra “địa chỉ vật lý có nằm trong RAM không”, test này sẽ thất bại về bảo mật dù không crash. Protection đúng phải gắn địa chỉ với identity của caller.

## 12.8 Walkthrough os\_mm64\_canonical: giải thích mọi con số

Test map bốn địa chỉ:

| Địa chỉ            | Indexes           | Kết quả                             |
|--------------------|-------------------|-------------------------------------|
| 0x0                | [0, 0, 0, 0, 0]   | Low canonical, tạo nhánh đầu tiên.  |
| 0x200000           | [0, 0, 0, 1, 0]   | Low canonical, khác PMD branch.     |
| 0xff00000000000000 | [256, 0, 0, 0, 0] | High canonical, tạo PGD branch mới. |
| 0x1000000000000000 | Không walk        | Non-canonical, bị từ chối.          |

Giải thích thống kê byte:

1. Khi khởi tạo `mm_struct`, root PGD chiếm 4096 byte.
2. Walk đầu tiên tại 0x0 cần thêm P4D, PUD, PMD và PT:  $4096 + 4 \times 4096 = 20480$  byte.
3. 0x200000 chia sẻ PGD/P4D/PUD/PMD table, nhưng cần một PT table mới dưới PMD index 1: tổng  $20480 + 4096 = 24576$  byte.
4. High canonical address đi sang PGD index 256, cần bốn bảng con mới:  $24576 + 4 \times 4096 = 40960$  byte.
5. Non-canonical address bị từ chối trước walk nên statistics không tăng.

## 13 Bài tính thực hành có lời giải

### 13.1 Bài tính scheduler 1: priority và quantum

**Đề bài.** Một CPU, quantum 2. Process A đến lúc 0, priority 2, có năm instruction. Process B đến lúc 1, priority 0, có ba instruction. Bỏ qua độ trễ loader. Hãy lập thứ tự dispatch theo policy của repository.

**Lời giải.**

1. Lúc 0 chỉ A sẵn sàng nên A được dispatch, chạy hai instruction.
2. B đến lúc 1 nhưng không preempt A giữa quantum.
3. Ở boundary kế tiếp, A được put lại  $Q_2$ ; B trong  $Q_0$  nên B được chọn.
4. B chạy hai instruction, hết quantum, rồi vẫn được chọn do  $Q_0$  có priority cao hơn.
5. B chạy instruction cuối và hoàn tất; A chạy ba instruction còn lại qua hai dispatch.

Dispatch order: A, B, B, A, A

### 13.2 Bài tính scheduler 2: weighted cycle

**Đề bài.** Ba queue luôn không rỗng:  $Q_{137}, Q_{138}, Q_{139}$ . Mỗi dispatch chạy đúng một quantum. Trong một weighted cycle, mỗi queue được chọn bao nhiêu lần?

**Lời giải.**

$$slot[137] = 3, \quad slot[138] = 2, \quad slot[139] = 1$$

Do scheduler quét từ priority nhỏ lên lớn, thứ tự là:

$$Q_{137}, Q_{137}, Q_{137}, Q_{138}, Q_{138}, Q_{139}$$

Sau đó mọi queue có PCB nhưng đều hết quota, scheduler reset slot và bắt đầu chu kỳ mới. Tỷ lệ CPU theo số quantum là 3 : 2 : 1.

### 13.3 Bài tính địa chỉ legacy

**Đề bài.** Page size 256 byte, virtual address 700. Page 2 map vào frame 9. Tính page number, offset và physical address.

**Lời giải.**

$$pgn = \left\lfloor \frac{700}{256} \right\rfloor = 2, \quad offset = 700 - 2 \times 256 = 188$$

$$paddr = 9 \times 256 + 188 = 2492$$

Nếu region kết thúc ở virtual address 700, access tại 700 vẫn phải bị từ chối do interval dùng dạng half-open  $[start, end)$ , dù page translation có thể tính ra một physical address.

### 13.4 Bài tính internal fragmentation

**Đề bài.** Cấp ba region lần lượt 100, 200 và 300 byte, mỗi region phải map số page 256 byte nguyên. Tính tổng page bytes và internal fragmentation nếu xét từng allocation độc lập.

$$\begin{aligned} 100 \rightarrow 256 &\Rightarrow 156 \text{ byte thừa} \\ 200 \rightarrow 256 &\Rightarrow 56 \text{ byte thừa} \\ 300 \rightarrow 512 &\Rightarrow 212 \text{ byte thừa} \end{aligned}$$

Tổng dữ liệu yêu cầu là 600 byte, tổng page bytes là 1024 byte, internal fragmentation là 424 byte. Repository có thể tận dụng phần dư alignment qua free-region list, nên fragmentation logic thực tế có thể thấp hơn cách tính độc lập này.

### 13.5 Bài tính MM64

**Đề bài.** Tách địa chỉ 0x200000 theo năm cấp.

$$0x200000 = 2^{21}$$

Bit 21 là bit thấp nhất của PMD index, nên:

$$PGD = 0, \quad P4D = 0, \quad PUD = 0, \quad PMD = 1, \quad PT = 0, \quad offset = 0$$

**Đề bài.** Vì sao  $0x1000000000000000$  không canonical?

Địa chỉ này đặt bit 56 bằng 1 nhưng các bit 63–57 vẫn bằng 0. Với bit 56 bằng 1, các bit trên bắt buộc đều bằng 1, nên địa chỉ vi phạm sign extension.

### 13.6 Bài mô phỏng swap từng bước

Giả sử RAM chỉ có hai frame  $F_0, F_1$ , process lần lượt truy cập ba page  $P_0, P_1, P_2$ , rồi quay lại  $P_0$ . FIFO ban đầu rỗng.

| Access | $F_0$ | $F_1$ | FIFO       | Hành động                                                     |
|--------|-------|-------|------------|---------------------------------------------------------------|
| $P_0$  | $P_0$ | trống | $P_0$      | Map page đầu.                                                 |
| $P_1$  | $P_0$ | $P_1$ | $P_0, P_1$ | Dùng frame còn trống.                                         |
| $P_2$  | $P_2$ | $P_1$ | $P_1, P_2$ | Victim FIFO là $P_0$ ; swap out $P_0$ , map $P_2$ vào $F_0$ . |
| $P_0$  | $P_2$ | $P_0$ | $P_2, P_0$ | Victim là $P_1$ ; swap out $P_1$ , swap in $P_0$ vào $F_1$ .  |

Tại mỗi lần thay thế, PTE victim chuyển từ present sang swapped; PTE target chuyển sang present và chứa FPN mới.

## 14 Các quyết định thiết kế và giới hạn cần nói thẳng

### 14.1 Các quyết định hợp lý

- MLQ bucket tách priority selection khỏi FIFO queue.
- Running list giúp multi-CPU ownership và PID lookup.
- Memory operation đi qua syscall 17 để tập trung kiểm tra quyền.
- Sparse MM64 page table tránh cấp toàn bộ cây.
- Semantic assertions kiểm tra các hành vi quan trọng thay vì chỉ exit code.

### 14.2 Các giới hạn kỹ thuật

- Queue dùng mảng và dịch phần tử nên dequeue là  $O(n)$ .
- PID lookup tuyến tính qua các queue.

- Một global `mmvm_lock` đơn giản nhưng hạn chế song song memory.
- FIFO replacement không phản ánh locality như LRU/Clock.
- Simulator không có TLB, hardware page fault, privilege mode thật hoặc context switch register đầy đủ.
- Một số cấu trúc legacy vẫn tồn tại bên cạnh MM64 để tương thích code nền.

### 14.3 Nếu được cải tiến

1. Dùng ring buffer cho FIFO queue và hash map cho PID registry.
2. Tách lock theo process/address space để tăng concurrency.
3. Thêm test stress đa CPU, invalid region ID, out-of-memory và swap đầy.
4. Thêm ASan/UBSan và race detector vào pipeline.
5. Tách rõ API page-size legacy và MM64 để giảm nhầm lẫn.
6. Dùng Clock hoặc LRU approximation cho replacement.

## 15 Bộ câu hỏi vấn đáp kèm đáp án

### 15.1 Câu hỏi nền tảng

**Câu hỏi:** *Process khác thread như thế nào?*

**Trả lời ngắn:** Process thường có address space riêng; các thread trong cùng process chia sẻ address space nhưng có execution context riêng. Trong simulator, mỗi CPU/loader là pthread của chương trình simulator, còn PCB đại diện cho process được mô phỏng.

**Câu hỏi:** *PCB cần thiết vì sao?*

**Trả lời ngắn:** Scheduler không thể chỉ lưu tên chương trình. Nó cần toàn bộ trạng thái để tạm dừng và tiếp tục: PID, PC, registers, priority và memory context.

**Câu hỏi:** *Virtual memory mang lại gì ngoài việc có nhiều bộ nhớ hơn RAM?*

**Trả lời ngắn:** Isolation, protection, address-space abstraction, sparse allocation, sharing có kiểm soát và khả năng swap. “Nhiều hơn RAM” chỉ là một lợi ích.

### 15.2 Câu hỏi scheduler

**Câu hỏi:** *Số priority nhỏ hơn hay lớn hơn được ưu tiên?*

**Trả lời ngắn:** Số nhỏ hơn được ưu tiên vì scheduler quét từ 0 lên 139.

**Câu hỏi:** *Weighted slot có đảm bảo không starvation không?*

**Trả lời ngắn:** Nó giảm starvation vì slot được reset và queue thấp vẫn có quota, nhưng thời gian chờ có thể rất dài nếu tải priority cao lớn. Đảm bảo hình thức còn phụ thuộc policy reset và việc queue thấp có liên tục sẵn sàng hay không.

**Câu hỏi:** *Tại sao phải purge running list khi put process?*

**Trả lời ngắn:** Nếu không purge, PCB vừa ở running list vừa ở ready queue. Kernel có thể nghĩ nó đang chạy trong khi scheduler dispatch nó lần nữa, phá vỡ single-ownership.

**Câu hỏi:** *Hai CPU có thể lấy cùng một PCB không?*

**Trả lời ngắn:** Không nếu `get_proc()` dequeue và enqueue running list trong cùng `queue_lock`. Nếu bỏ lock, điều đó hoàn toàn có thể xảy ra.

### 15.3 Câu hỏi synchronization

**Câu hỏi:** *Condition variable khác mutex thế nào?*

**Trả lời ngắn:** Mutex bảo vệ độc quyền critical section. Condition variable giúp thread ngủ chờ một điều kiện và phải được dùng cùng mutex để kiểm tra/cập nhật trạng thái an toàn.

**Câu hỏi:** *Tại sao condition wait thường nằm trong vòng while, không phải if?*

**Trả lời ngắn:** Vì có thể có spurious wakeup hoặc thread khác thay đổi điều kiện trước khi thread vừa thức dậy lại mutex. Vòng while kiểm tra lại predicate.

**Câu hỏi:** *System call chạy quá lâu thì OS thật xử lý thế nào?*

**Trả lời ngắn:** Kernel có thể theo dõi timer/watchdog, cho phép preemption ở các điểm an toàn, ngắt operation hoặc đánh dấu hung task tùy loại syscall. Không thể tùy tiện dừng giữa critical section mà không bảo toàn dữ liệu.

### 15.4 Câu hỏi memory

**Câu hỏi:** *Internal và external fragmentation là gì?*

**Trả lời ngắn:** Internal fragmentation là phần thừa bên trong block đã cấp, ví dụ allocation 100 byte chiếm page 256 byte. External fragmentation là tổng dung lượng trống đủ nhưng bị chia thành nhiều khoảng rời nên không tạo được block liên tiếp.

**Câu hỏi:** *Vì sao free region có thể tái sử dụng mà chưa cần unmap page?*

**Trả lời ngắn:** Free region là quản lý allocation logic bên trong VMA. Page vật lý có thể vẫn map cho process và được dùng lại nhanh. Unmap/trả frame ngay là policy khác, tiết kiệm RAM hơn nhưng phức tạp hơn.

**Câu hỏi:** *Tại sao raw physical I/O nguy hiểm?*

**Trả lời ngắn:** Physical address bỏ qua isolation của virtual memory. Nếu user chọn tùy ý physical address, nó có thể đọc/ghi dữ liệu process khác hoặc kernel.

**Câu hỏi:** *Page fault trong simulator khác OS thật ra sao?*

**Trả lời ngắn:** Ở OS thật, MMU phát exception khi PTE không present; kernel fault handler xử lý. Trong simulator, code chủ động kiểm tra PTE trong `pg_getpage()` và thực hiện swap bằng hàm C.

## 15.5 Câu hỏi MM64

**Câu hỏi:** Vì sao 512 entry mỗi cấp tương ứng 9 bit?

**Trả lời ngắn:** Vì  $512 = 2^9$ , nên 9 bit đủ chọn một entry.

**Câu hỏi:** Vì sao một page-table page có kích thước 4096 byte?

**Trả lời ngắn:** 512 entry nhân 8 byte mỗi entry bằng 4096 byte, đúng một page 4 KiB.

**Câu hỏi:** Sparse allocation tiết kiệm trong trường hợp nào?

**Trả lời ngắn:** Khi process chỉ dùng một phần nhỏ, rời rạc của address space lớn. Nếu toàn bộ address space được map dày đặc, lợi ích giảm và số bảng vẫn rất lớn.

**Câu hỏi:** TLB dùng để làm gì?

**Trả lời ngắn:** TLB cache kết quả virtual-page sang physical-frame, giúp phần lớn access không phải walk nhiều cấp. Simulator hiện chưa mô phỏng TLB.

## 15.6 Câu hỏi về kiểm thử và báo cáo

**Câu hỏi:** Tại sao test output không nhất thiết giống từng dòng?

**Trả lời ngắn:** Nhiều thread làm thứ tự các sự kiện độc lập thay đổi. Điều cần giữ là các invariant: không trùng PID trên hai CPU, priority được tôn trọng ở boundary, protection bị từ chối đúng, dữ liệu roundtrip không đổi.

**Câu hỏi:** Làm sao chứng minh code không chỉ “chạy được” mà còn đúng?

**Trả lời ngắn:** Lập traceability từ yêu cầu đến invariant, từ invariant đến testcase, rồi dùng semantic assertion kiểm tra bằng chứng cụ thể. Đồng thời phân tích đường lỗi và giới hạn.

## 16 Phiên vấn đáp mô phỏng hoàn chỉnh

### 16.1 Cách trả lời một câu vấn đáp kỹ thuật

Một câu trả lời mạnh thường có bốn lớp, theo thứ tự:

1. **Định nghĩa:** nói đúng khái niệm trong một hoặc hai câu.
2. **Liên hệ implementation:** chỉ ra cấu trúc/hàm/file cụ thể.
3. **Bảng chứng:** nêu testcase hoặc output chứng minh.
4. **Trade-off:** nói giới hạn và hướng cải tiến nếu được hỏi sâu.

Ví dụ, thay vì chỉ nói “nhóm dùng mutex để tránh race condition”, câu trả lời tốt hơn là: “Các MLQ bucket và running list là shared mutable state của loader cùng nhiều CPU. Nhóm bảo vệ toàn bộ thao tác dequeue–enqueue chuyển ownership bằng `queue_lock` trong `src/sched.c`. Test `sched_2_cpu` cho thấy hai CPU không dispatch cùng PID. Global lock đơn giản nhưng có thể trở thành bottleneck khi tăng số CPU.”

## 16.2 Vòng 1: trình bày tổng quan trong hai phút

**Câu hỏi:** *Em hãy mô tả kiến trúc bài làm từ khi chạy binary đến khi process kết thúc.*

**Trả lời ngắn:** `main()` đọc configuration, khởi tạo timer, scheduler, MEMRAM, MEM-SWAP, loader và các CPU thread. Loader parse process file thành PCB, gán kernel handle và memory context rồi thêm PCB vào MLQ tại thời điểm cấu hình. Mỗi CPU lấy PCB qua `get_proc()`, chạy từng instruction trong `run()`, và trả PCB về queue khi hết quantum. Memory instruction đi qua library và unified syscall 17 để kernel lookup caller bằng PID, kiểm tra quyền, rồi thao tác VMA, page table hoặc physical memory. Timer đồng bộ mọi thread theo time slot. Khi `pc == code->size`, CPU xóa PCB khỏi running list, giải phóng address space và lấy process khác.

**Câu hỏi:** *Ba phần khó nhất của bài là gì?*

**Trả lời ngắn:** Thứ nhất là giữ ownership PCB đúng khi nhiều CPU cùng scheduling. Thứ hai là làm memory operation đi qua kernel interface mà vẫn xác định đúng caller và bảo vệ physical frame. Thứ ba là page table MM64 sparse allocation phải vừa kiểm tra canonical address vừa quản lý cấp/giải phóng các bảng con.

## 16.3 Vòng 2: đào sâu scheduler và synchronization

**Câu hỏi:** *Scheduler của nhóm có phải strict priority scheduling không?*

**Trả lời ngắn:** Không hoàn toàn. Khi chọn process, scheduler quét priority từ cao đến thấp, nhưng mỗi queue có weighted slot. Queue priority cao hết quota sẽ tạm nhường queue thấp hơn cho đến khi reset chu kỳ. Ngoài ra process mới priority cao chỉ được chọn tại boundary quantum, không preempt tức thời.

**Câu hỏi:** *Nếu hai CPU cùng gọi `get_proc()` thì điều gì đảm bảo chúng không lấy cùng PCB?*

**Trả lời ngắn:** Cả quá trình chọn bucket, dequeue PCB và enqueue vào running list nằm trong một critical section bảo vệ bởi `queue_lock`. CPU thứ hai chỉ thấy queue sau khi PCB đã bị loại khỏi ready queue.

**Câu hỏi:** *Tại sao running list cần thiết nếu PCB đã nằm trong biến cục bộ của CPU thread?*

**Trả lời ngắn:** Biến cục bộ chỉ CPU đó thấy. Kernel subsystem khác cần registry để lookup caller theo PID khi syscall chạy, và hệ thống cần biết PCB nào đang được giữ khỏi ready queue. Running list cung cấp trạng thái kernel-visible đó.

**Câu hỏi:** *Timer của nhóm có phải hardware interrupt không?*

**Trả lời ngắn:** Không. Đây là pthread dùng condition variables để tạo barrier theo time slot. Nó mô phỏng nhịp thời gian và sự phối hợp thiết bị, nhưng không thể ngắt CPU thread tùy ý như hardware timer interrupt thật.

**Câu hỏi:** *Có deadlock nào tiềm ẩn không?*

**Trả lời ngắn:** Thiết kế hiện tại giảm nguy cơ bằng cách dùng lock theo subsystem và không giữ `queue_lock` khi chạy instruction. Tuy nhiên khi mở rộng, nếu một đường code giữ `mmvm_lock` rồi chờ `memphy_lock`, còn đường khác giữ `memphy_lock` rồi chờ `mmvm_lock`, deadlock có thể xảy ra. Cần quy định lock ordering và tránh gọi operation dài trong critical



section.

## 16.4 Vòng 3: đào sâu memory và protection

**Câu hỏi:** *ALLOC 100 byte có nhất thiết chỉ chiếm đúng 100 byte RAM không?*

**Trả lời ngắn:** Không. Region logic dài 100 byte, nhưng mapping vật lý được align theo page size nên có thể cần một page 256 byte. Phần dư có thể được đưa vào free-region list để tái sử dụng. Cần phân biệt kích thước region với dung lượng frame đã map.

**Câu hỏi:** *Tại sao FREE không luôn trả frame ngay cho RAM?*

**Trả lời ngắn:** Implementation ưu tiên reuse region trong cùng VMA. Nó xóa symbol region và đưa khoảng trống vào free list; page mapping có thể giữ lại. Cách này đơn giản và allocation lại nhanh, nhưng không giảm resident memory tức thời.

**Câu hỏi:** *Nếu PID 2 biết chính xác physical address của PID 1 thì sao?*

**Trả lời ngắn:** Biết địa chỉ không tạo ra quyền. Syscall physical I/O lookup PCB PID 2 và kiểm tra frame có thuộc page table của PID 2 không. Nếu không, kernel từ chối. Test `os_cross_pid_protection` minh chứng trường hợp frame 0.

**Câu hỏi:** *Tại sao không cho user truyền thẳng `pcb_t *` vào syscall?*

**Trả lời ngắn:** User có thể giả mạo con trỏ sang PCB khác hoặc con trỏ không hợp lệ. Kernel phải xây dựng identity từ trạng thái do kernel sở hữu. Repository dùng PID và `find_proc_by_pid()` để mô phỏng nguyên tắc đó.

**Câu hỏi:** *Điều gì xảy ra khi page không present?*

**Trả lời ngắn:** `pg_getpage()` chọn victim FIFO, lấy swap frame, copy victim từ RAM sang SWAP qua syscall, cập nhật victim PTE thành swapped, đưa target page về victim frame nếu cần, cập nhật target PTE present và thêm target vào FIFO.

## 16.5 Vòng 4: đào sâu MM64

**Câu hỏi:** *Tại sao địa chỉ 57-bit vẫn lưu trong kiểu 64-bit?*

**Trả lời ngắn:** CPU/API dùng thanh ghi 64-bit, nhưng phần cứng chỉ hiện thực 57 bit virtual address. Các bit trên phải là sign extension để tạo canonical address. Kiểu 64-bit vẫn cần thiết để chứa cả low và high canonical ranges.

**Câu hỏi:** *Tại sao `0xff00000000000000` có PGD index 256?*

**Trả lời ngắn:** Bit 56 bằng 1 và các bit 63–57 đều bằng 1 nên địa chỉ canonical cao. Trường PGD là bits 56–48; giá trị của trường này là 256, còn bốn index dưới bằng 0.

**Câu hỏi:** *Tại sao first mapping báo 20.480 byte thay vì 16.384 byte?*

**Trả lời ngắn:** Root PGD 4096 byte đã được cấp khi khởi tạo `mm_struct`. Walk đầu tiên cấp thêm bốn bảng con P4D, PUD, PMD và PT, mỗi bảng 4096 byte. Tổng là  $5 \times 4096 = 20480$ .

**Câu hỏi:** *Nếu dùng page table phẳng cho 57-bit address và page 4 KiB thì cần bao nhiêu PTE?*

**Trả lời ngắn:** Có  $57 - 12 = 45$  bit page number, tức  $2^{45}$  PTE. Với 8 byte/PTE, dung

lượng lý thuyết là  $2^{48}$  byte, tức 256 TiB cho một page table phẳng. Đó là lý do hierarchical sparse paging cần thiết.

## 16.6 Vòng 5: phản biện kiểm thử và chất lượng

**Câu hỏi:** *Nhóm nói test pass. Điều đó có đủ chứng minh scheduler đúng không?*

**Trả lời ngắn:** Không. Một tập test hữu hạn chỉ tăng độ tin cậy cho các trường hợp đã bao phủ. Test hiện chứng minh priority boundary, weighted quota và multi-CPU ownership ở các scenario cụ thể. Vẫn cần stress test, random schedule, assertion nội bộ và race detector để tìm lỗi concurrency hiếm.

**Câu hỏi:** *Điểm yếu lớn nhất của test hiện tại là gì?*

**Trả lời ngắn:** Chưa có stress test dài cho nhiều CPU/process, chưa ép rõ các đường out-of-memory/swap-full, chưa kiểm tra mọi invalid argument và chưa dùng công cụ phát hiện data race. Output assertion hiện chủ yếu kiểm tra các mốc bắt buộc, chưa kiểm tra đầy đủ quan hệ thời gian.

**Câu hỏi:** *Nếu output chạy hai lần khác thứ tự thì có phải bug không?*

**Trả lời ngắn:** Không nhất thiết. Các sự kiện độc lập giữa pthread có thể đổi thứ tự. Bug chỉ khi invariant bị phá, ví dụ cùng PID được dispatch trên hai CPU, một process biến mất khỏi mọi queue, hoặc priority policy sai tại scheduling boundary.

## 16.7 Các câu hỏi bấy thường gặp

| Câu hỏi/bẫy                                 | Cách trả lời chính xác                                                                                                                                                                    |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Priority 0 luôn chạy ngay lập tức?          | Không. Nó được ưu tiên ở lần scheduling tiếp theo; CPU hiện tại chạy đến hết quantum hoặc hoàn tất.                                                                                       |
| FREE luôn giải phóng physical frame?        | Không trong user-region path hiện tại; nó chủ yếu trả logical region cho free list.                                                                                                       |
| Có page table nghĩa là access được phép?    | Không. Page table hỗ trợ translation; kernel vẫn phải kiểm tra ownership/protection.                                                                                                      |
| Mutex làm chương trình deterministic?       | Không. Mutex bảo toàn critical section, nhưng thứ tự thread hợp lệ vẫn có thể khác nhau.                                                                                                  |
| MM64 dùng page size 4096 ở mọi memory path? | Không nên khẳng định như vậy. MM64 hierarchy dùng offset 12 bit để khảo sát bảng năm cấp, trong khi nhiều allocation/PTE path của simulator vẫn dựa trên <code>PAGING_PAGESZ=256</code> . |
| Test pass nghĩa là không còn bug?           | Không. Nó chỉ chứng minh các property được assert trong các input đã chạy.                                                                                                                |

## 17 Bài tập tự luyện

### 17.1 Mức 1: đọc và dự đoán

1. Với quantum 2, một process có 5 lệnh `calc` cần ít nhất bao nhiêu lần dispatch nếu không có process khác?
2. Tính page number và offset của virtual address 700 với page size 256.
3. Region  $[16, 32)$  cho phép offset nào? Offset 16 có hợp lệ không?
4. Với priority 5, initial weighted slot là bao nhiêu?
5. Viết năm index MM64 của địa chỉ `0x200000`.

### 17.2 Mức 2: giải thích bằng source

1. Lần theo ALLOC từ `src/cpu.c` đến nơi frame được lấy khỏi free list. Ghi tên mọi hàm trung gian.
2. Chỉ ra lock nào bảo vệ từng tài nguyên: scheduler queue, VMA metadata, free-frame list và timer event.
3. Giải thích vì sao `find_proc_by_pid()` phải duyệt running list.
4. Chỉ ra nơi non-canonical MM64 address bị từ chối.
5. Chỉ ra nơi frame được trả khi process kết thúc.

### 17.3 Mức 3: thiết kế testcase

1. Thiết kế testcase chứng minh READ offset đúng bằng region size bị từ chối.
2. Thiết kế testcase tạo nhiều process cùng priority để kiểm tra FIFO.
3. Thiết kế testcase làm SWAP đầy và nêu output mong đợi.
4. Thiết kế testcase kiểm tra double-free.
5. Thiết kế testcase để phát hiện hai CPU dispatch cùng PID.

### 17.4 Đáp án nhanh mức 1

1. Ba lần dispatch:  $2 + 2 + 1$  instruction.
2.  $p_{gn} = \lfloor 700/256 \rfloor = 2$ ,  $offset = 700 \bmod 256 = 188$ .
3. Offset hợp lệ là 0–15; offset 16 không hợp lệ.
4.  $140 - 5 = 135$ .
5.  $0x200000 = 2^{21}$ , nên PMD index bằng 1; các index còn lại bằng 0.

## 17.5 Gọi ý lời giải mức 2

- Đường ALLOC chính đi qua bốn tầng:
  - CPU/library: `run()` đến `liballoc()`, rồi `mem_syscall()`.
  - Dispatcher: `_syscall()` đến `__sys_memmap()`.
  - VMA mapping: `__alloc()` đến `inc_vma_limit()`, rồi `vm_map_range()`.
  - Physical allocation: `alloc_pages_range()` đến `MEMPHY_get_freefp()`.
- Scheduler queue dùng `queue_lock`; VMA/region metadata dùng `mmvm_lock`; free-frame list dùng `memphy_lock`; timer event dùng từng cặp `event_lock/timer_lock`.
- PCB đang gọi syscall không còn ở ready queue; nó nằm trong running list nên PID lookup phải duyệt danh sách này.
- Hàm `paging64_is_canonical()` được gọi trước khi tách index/page-table walk. Hàm này cũng được gọi trong `vmap_pgd_memset()` trước khi map.
- Khi process kết thúc, `cpu_routine()` gọi `destroy_mm()`; MM64 release path trả user frames và giải phóng bảng con.

## 17.6 Gọi ý lời giải mức 3

**Test offset đúng bằng size.** Tạo region 16 byte rồi `read region 16 dst`. Assertion mong đợi không có dòng read thành công và handler trả lỗi, vì offset hợp lệ lớn nhất là 15.

**Test FIFO cùng priority.** Tạo ba process cùng start time và priority, mỗi process có một hoặc hai instruction, một CPU. Kiểm tra thứ tự dispatch theo thứ tự loader enqueue. Để tránh phụ thuộc concurrency loader/CPU, có thể cho chúng vào cùng queue trước khi CPU bắt đầu hoặc dùng start times kiểm soát.

**Test SWAP đầy.** Dùng RAM rất nhỏ, SWAP chỉ đủ một frame, rồi truy cập nhiều page buộc hai lần swap out khi slot cũ chưa giải phóng. Mong đợi memory operation trả lỗi sạch, không crash và không làm hỏng PTE hiện có.

**Test double-free.** Cấp region, free hai lần. Lần đầu thành công; lần hai phải trả lỗi vì symbol region đã được reset thành `[0, 0)`. Free list không được chứa hai node trùng nhau.

**Test duplicate dispatch.** Chạy nhiều CPU và ít process đủ dài; phân tích log theo slot hoặc thêm assertion runtime rằng một PID chỉ xuất hiện một lần trong running ownership set.

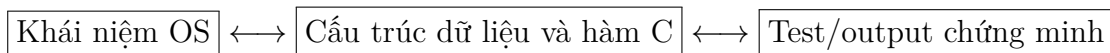
## 18 Cheat sheet trước khi trình bày

| Chủ đề    | Một câu phải nhớ                                                                                                        |
|-----------|-------------------------------------------------------------------------------------------------------------------------|
| Kiến trúc | Loader tạo PCB; scheduler chọn PCB; CPU chạy instruction; syscall đưa thao tác nhảy cảm vào kernel; timer đồng bộ slot. |

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| MLQ             | 140 FIFO queues; số priority nhỏ hơn cao hơn; weighted slot bằng 140 – <i>prio</i> . |
| Quantum         | Giới hạn số instruction process giữ CPU trước khi put lại queue.                     |
| Synchronization | Queue, VMA và free-frame list là shared mutable state nên phải lock.                 |
| Virtual memory  | Region/VMA tổ chức địa chỉ logic; page table map virtual page sang physical frame.   |
| Protection      | Kernel lookup caller bằng PID và từ chối physical frame không thuộc caller.          |
| Swap            | Khi page không present, chọn victim FIFO, swap out victim và đưa target về RAM.      |
| MM64            | 57 bit = năm index 9 bit + offset 12 bit; địa chỉ phải canonical.                    |
| Kiểm thử        | Build/pass chưa đủ; semantic assertion phải chứng minh invariant.                    |

## 19 Kết luận học tập

Để thực sự làm chủ bài tập, hãy luôn nói ba tầng:



Nếu chỉ nhớ khái niệm, ta khó giải thích code. Nếu chỉ nhớ code, ta khó trả lời trade-off. Nếu chỉ nhớ output, ta không chứng minh được vì sao kết quả đúng. Khi liên kết cả ba, toàn bộ repository trở thành một mô hình nhất quán: scheduler bảo toàn quyền sở hữu CPU, virtual memory bảo toàn isolation, system call bảo toàn ranh giới đặc quyền, và synchronization bảo toàn trạng thái dùng chung.